

T2 Análisis de componentes principales (PCA)

Índice

1. [Introducción](#)
2. [Análisis de componentes principales \(PCA\)](#)
3. [PCA con la EVD](#)
4. [PCA con la SVD](#)

Anexo

- [Conceptos básicos y notación](#)

Cita

Juan Ciscar, Alfons. "Material docente de la asignatura [Aprendizaje Automático \(APA\)](#)". [Grado de Inteligencia Artificial \(GIAR\)](#), [Escuela Técnica Superior de Ingeniería Informática \(ETSINF\)](#), [Dep. de Sistemas Informáticos y Computación \(DSIC\)](#), [Universitat Politècnica de València \(UPV\)](#), primer semestre del curso 2026/27.

1 Introducción

Objetivo principal: aplicar PCA para reducir la dimensión de los datos en tareas de aprendizaje automático

Algebra lineal: conceptos básicos y notación en anexo

PCA: escoge una proyección lineal de mínima **distorsión o error de reconstrucción**

PCA con la EVD: a partir de la matriz de covarianzas empírica

PCA con la SVD: a partir de la matriz de datos centrada; más eficiente

2 Análisis de componentes principales (PCA)

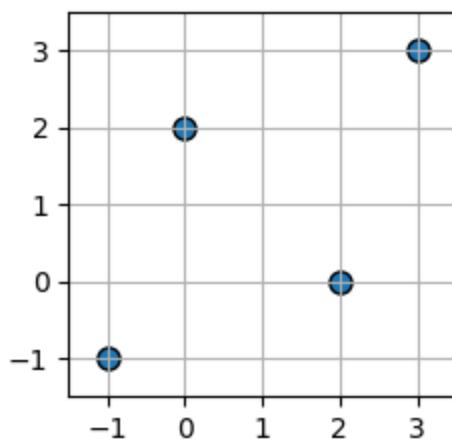
Maldición de la dimensionalidad: muchas técnicas de ML empeoran sensiblemente con datos de alta dimensión

Reducción de la dimensión: dada una matriz de N datos en $\mathbb{R}^D$, queremos aprender (no supervisadamente) una transformación de $\mathbb{R}^D$ en $\mathbb{R}^K$, $K \ll D$, que "aproxime bien" los datos originales

Ejemplo: matriz de $N = 4$ datos de $D = 2$ dimensiones a reducir a $K = 1$

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1^t \\ \mathbf{x}_2^t \\ \mathbf{x}_3^t \\ \mathbf{x}_4^t \end{pmatrix} = \begin{pmatrix} -1 & -1 \\ 0 & 2 \\ 2 & 0 \\ 3 & 3 \end{pmatrix}$$

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt; X = np.array([ [-1, -1], [0, 2], [2, 0], [3, 3] ]); N=len(X)
fig, ax = plt.subplots(figsize=(2.5, 2.5)); plt.grid(True); plt.axis([-1.5, 3.5, -1.5, 3.5])
plt.xticks(range(-1,4)); plt.yticks(range(-1,4)); plt.scatter(*X.T, facecolor='C0', edgecolor='k', s=64);
```



Codificación y decodificación: operaciones de reducción de la dimensión y reconstrucción

$$z = \text{encode}(x) \quad \text{y} \quad \hat{x} = \text{decode}(z), \quad x, \hat{x} \in \mathbb{R}^D, \quad z \in \mathbb{R}^K$$

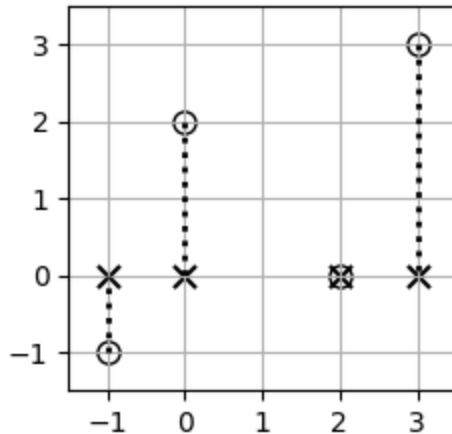
Codificación y decodificación lineales: con $\mathbf{W} \in \mathbb{R}^{D \times K}$

$$z = \mathbf{W}^t x \quad \text{y} \quad \hat{x} = \mathbf{W} z \quad \text{o, con matrices,} \quad \mathbf{Z} = \mathbf{XW} \quad \text{y} \quad \hat{\mathbf{X}} = \mathbf{ZW}^t$$

Ejemplo (cont.): $\mathbf{W} = (1, 0)^t$

$$\mathbf{Z} = \begin{pmatrix} -1 & -1 \\ 0 & 2 \\ 2 & 0 \\ 3 & 3 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} -1 \\ 0 \\ 2 \\ 3 \end{pmatrix} \quad \hat{\mathbf{X}} = \begin{pmatrix} -1 \\ 0 \\ 2 \\ 3 \end{pmatrix} (1, 0) = \begin{pmatrix} -1 & 0 \\ 0 & 0 \\ 2 & 0 \\ 3 & 0 \end{pmatrix}$$

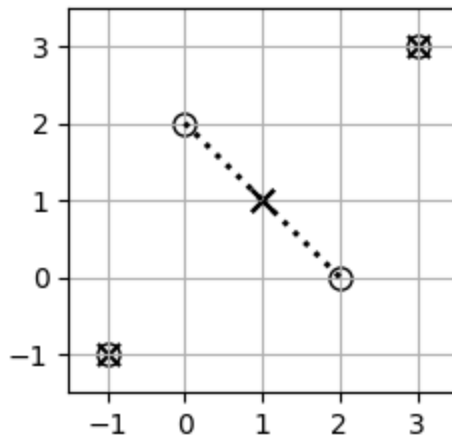
```
In [ ]: from matplotlib.collections import LineCollection; fig, ax = plt.subplots(figsize=(2.5, 2.5))
plt.grid(True); plt.axis([-1.5, 3.5, -1.5, 3.5]); plt.xticks(range(-1,4)); plt.yticks(range(-1,4))
plt.scatter(*X.T, facecolor='white', edgecolor='k', s=64); K = 1; W = np.array([1, 0]).reshape(-1, K)
Z = (X @ W).reshape(-1, K); hX = Z @ W.T; plt.scatter(*hX.T, facecolor='black', s=64, marker='x')
lines = np.zeros((N, 2, 2)); lines[:, 0, :] = X; lines[:, 1, :] = hX
ax.add_collection(LineCollection(lines, linewidths=2, colors='black', linestyle='dotted'));
```



Ejemplo (cont.): $\mathbf{W} = (\sqrt{2}/2, \sqrt{2}/2)^t$

$$\mathbf{Z} = \begin{pmatrix} -1 & -1 \\ 0 & 2 \\ 2 & 0 \\ 3 & 3 \end{pmatrix} \begin{pmatrix} \sqrt{2}/2 \\ \sqrt{2}/2 \end{pmatrix} = \begin{pmatrix} -\sqrt{2} \\ \sqrt{2} \\ \sqrt{2} \\ 3\sqrt{2} \end{pmatrix} \quad \hat{\mathbf{X}} = \begin{pmatrix} -\sqrt{2} \\ \sqrt{2} \\ \sqrt{2} \\ 3\sqrt{2} \end{pmatrix} (\sqrt{2}/2, \sqrt{2}/2) = \begin{pmatrix} -1 & -1 \\ 1 & 1 \\ 1 & 1 \\ 3 & 3 \end{pmatrix}$$

```
In [ ]: from matplotlib.collections import LineCollection; fig, ax = plt.subplots(figsize=(2.5, 2.5))
plt.grid(True); plt.axis([-1.5, 3.5, -1.5, 3.5]); plt.xticks(range(-1,4)); plt.yticks(range(-1,4))
plt.scatter(*X.T, facecolor='white', edgecolor='k', s=64)
K = 1; W = np.array([np.sqrt(2)/2, np.sqrt(2)/2]).reshape(-1, K)
Z = (X @ W).reshape(-1, K); hX = Z @ W.T; plt.scatter(*hX.T, facecolor='black', s=64, marker='x')
lines = np.zeros((N, 2, 2)); lines[:, 0, :] = X; lines[:, 1, :] = hX
ax.add_collection(LineCollection(lines, linewidths=2, colors='black', linestyle='dotted'));
```



Pregunta: ¿cómo escogerías $\mathbf{W}$?

PCA: escoge una proyección lineal ortogonal $\mathbf{W} \in \mathbb{R}^{D \times K}$ de mínima **distorsión o error de reconstrucción**

$$\begin{aligned}\mathcal{L}(\mathbf{W}) &= \frac{1}{N} \sum_n \|\mathbf{x}_n - \text{decode}(\text{encode}(\mathbf{x}_n))\|_2^2 \quad \text{con} \quad \text{encode}(\mathbf{x}) = \mathbf{W}^t \mathbf{x} \quad \text{y} \quad \text{decode}(\mathbf{z}) = \mathbf{W} \mathbf{z} \\ &= \frac{1}{N} \|\mathbf{X} - \hat{\mathbf{X}}\|_F^2 \quad \text{con} \quad \hat{\mathbf{X}} = \mathbf{Z} \mathbf{W}^t, \quad \mathbf{Z} = \mathbf{X} \mathbf{W}, \quad \|\mathbf{X} - \hat{\mathbf{X}}\|_F = \|\text{vec}(\mathbf{X} - \hat{\mathbf{X}})\|_2\end{aligned}$$

Ejemplo (cont.):

$$\mathcal{L}((1, 0)^t) = \frac{1}{4} \left\| \begin{pmatrix} -1 & -1 \\ 0 & 2 \\ 2 & 0 \\ 3 & 3 \end{pmatrix} - \begin{pmatrix} -1 & 0 \\ 0 & 0 \\ 2 & 0 \\ 3 & 0 \end{pmatrix} \right\|_F^2 = \frac{1}{4} \left\| \begin{pmatrix} 0 & -1 \\ 0 & 2 \\ 0 & 0 \\ 0 & 3 \end{pmatrix} \right\|_F^2 = \frac{14}{4} = 3.5$$

$$\mathcal{L}((\sqrt{2}/2, \sqrt{2}/2)^t) = \frac{1}{4} \left\| \begin{pmatrix} -1 & -1 \\ 0 & 2 \\ 2 & 0 \\ 3 & 3 \end{pmatrix} - \begin{pmatrix} -1 & -1 \\ 1 & 1 \\ 1 & 1 \\ 3 & 3 \end{pmatrix} \right\|_F^2 = \frac{1}{4} \left\| \begin{pmatrix} 0 & 0 \\ -1 & 1 \\ 1 & -1 \\ 0 & 0 \end{pmatrix} \right\|_F^2 = \frac{4}{4} = 1$$

Pregunta: $(\sqrt{2}/2, \sqrt{2}/2)^t$ produce menor distorsión que $(1, 0)^t$ pero, ¿es de mínima distorsión (PCA)?

3 PCA con la EVD

Media empírica y datos centrados: $\bar{\mathbf{x}}^t = \frac{1}{N} \mathbf{1}_N^t \mathbf{X}$ $\tilde{\mathbf{X}} = \mathbf{X} - \mathbf{1}_N \bar{\mathbf{x}}^t$

Ejemplo (cont.):

$$\bar{\mathbf{x}}^t = \frac{1}{4} (1, 1, 1, 1) \begin{pmatrix} -1 & -1 \\ 0 & 2 \\ 2 & 0 \\ 3 & 3 \end{pmatrix} = (1, 1) \quad \tilde{\mathbf{X}} = \begin{pmatrix} -1 & -1 \\ 0 & 2 \\ 2 & 0 \\ 3 & 3 \end{pmatrix} - \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} -2 & -2 \\ -1 & 1 \\ 1 & -1 \\ 2 & 2 \end{pmatrix}$$

Matriz de covarianzas empírica: $\Sigma = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^t = \frac{1}{N} \tilde{\mathbf{X}}^t \tilde{\mathbf{X}}$

Ejemplo (cont.):

$$\Sigma = \frac{1}{4} \begin{pmatrix} -2 & -1 & 1 & 2 \\ -2 & 1 & -1 & 2 \end{pmatrix} \begin{pmatrix} -2 & -2 \\ -1 & 1 \\ 1 & -1 \\ 2 & 2 \end{pmatrix} = \frac{1}{4} \begin{pmatrix} 10 & 6 \\ 6 & 10 \end{pmatrix} = \begin{pmatrix} 5/2 & 3/2 \\ 3/2 & 5/2 \end{pmatrix}$$

```
In [ ]: import numpy as np; X = np.array([ [-1, -1], [0, 2], [2, 0], [3, 3] ])
        S = np.cov(X, rowvar=False, bias=True); S
```

```
Out[ ]: array([[2.5, 1.5],
               [1.5, 2.5]])
```

Descomposición propia (eigenvector decomposition, EVD) de Σ : por ser simétrica y (semi-)definida positiva,

$$\Sigma = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^t \quad \text{con} \quad \mathbf{U} = (\mathbf{u}_1, \dots, \mathbf{u}_D) \text{ ortogonal} \quad \text{y} \quad \mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_D), \lambda_1 \geq \dots \geq \lambda_D \geq 0$$

Cálculo a mano: con la **ecuación característica** $|\lambda \mathbf{I}_D - \Sigma| = 0$ y escogiendo vectores propios normalizados

Ejemplo (cont.):

$$|\lambda \mathbf{I} - \Sigma| = 0 \rightarrow \left| \lambda \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - \begin{bmatrix} \frac{5}{2} & \frac{3}{2} \\ \frac{3}{2} & \frac{5}{2} \end{bmatrix} \right| = \left| \begin{bmatrix} \lambda - \frac{5}{2} & -\frac{3}{2} \\ -\frac{3}{2} & \lambda - \frac{5}{2} \end{bmatrix} \right| = 0$$

$$\rightarrow \left(\lambda - \frac{5}{2} \right)^2 - \frac{9}{4} = 0 \rightarrow \lambda^2 - 5\lambda + 4 = 0 \rightarrow \begin{cases} \lambda_1 = 4 \\ \lambda_2 = 1 \end{cases}$$

$$\Sigma \mathbf{e}_1 = \lambda_1 \mathbf{e}_1 \rightarrow \mathbf{e}_1 = (\alpha, \alpha)^t \xrightarrow{\|\mathbf{e}_1\|=1} \mathbf{e}_1 = (\sqrt{2}/2, \sqrt{2}/2)^t = (0.7071, 0.7071)^t$$

$$\Sigma \mathbf{e}_2 = \lambda_2 \mathbf{e}_2 \rightarrow \mathbf{e}_2 = (-\alpha, \alpha)^t \xrightarrow{\|\mathbf{e}_2\|=1} \mathbf{e}_2 = (-\sqrt{2}/2, \sqrt{2}/2)^t = (-0.7071, 0.7071)^t$$

$$\Sigma = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^t \quad \mathbf{U} = [\mathbf{e}_1, \mathbf{e}_2] \quad \mathbf{\Lambda} = \text{diag}(\lambda_1, \lambda_2)$$

```
In [ ]: La, U = np.linalg.eigh(S); i = La.argsort()[::-1]; La = La[i]; U = U[:,i]; Uq
```

```
Out[ ]: array([[ 0.70710678, -0.70710678],
               [ 0.70710678,  0.70710678]])
```

PCA: las K primeras columnas de $\mathbf{U}$ constituyen una proyección lineal de mínima distorsión

$$\mathbf{W}_{\text{pca}} = (\mathbf{u}_1, \dots, \mathbf{u}_K) \in \mathbb{R}^{D \times K}$$

Ejemplo (cont.): $\mathbf{W}_{\text{pca}} = (\sqrt{2}/2, \sqrt{2}/2)^t$

4 PCA con la SVD

Singular value decomposition (SVD) de los datos: asumimos **datos centrados** y $N \geq D$

$$\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^t = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^t + \cdots + \sigma_D \mathbf{u}_D \mathbf{v}_D^t$$

- **Vectores singulares izquierdos:** $\mathbf{U} \in \mathbf{R}^{N \times N}$, de columnas ortonormales, $\mathbf{U}^t \mathbf{U} = \mathbf{I}$
- **Valores singulares:** $\mathbf{S} \in \mathbf{R}^{N \times D}$ con valores $\sigma_i \geq 0$ en su diagonal; ceros fuera
- **Vectores singulares derechos:** $\mathbf{V} \in \mathbf{R}^{D \times D}$, de filas y columnas ortonormales, $\mathbf{V}^t \mathbf{V} = \mathbf{V} \mathbf{V}^t = \mathbf{I}$

SVD K -truncada: aproximación de rango K construida a partir de sus K mayores valores singulares, junto con sus K vectores singulares asociados, izquierdos y derechos

$$\hat{\mathbf{X}}_K = \mathbf{U}_K \mathbf{S}_K \mathbf{V}_K^t$$

PCA: las K primeras columnas de $\mathbf{V}$ constituyen una proyección lineal de mínima distorsión

$$\mathbf{W}_{\text{pca}} = \mathbf{V}_K \in \mathbb{R}^{D \times K}$$

Ejemplo (cont.):

```
In [ ]: import numpy as np; X = np.array([ [-1, -1], [0, 2], [2, 0], [3, 3] ])
X = X - np.mean(X, axis=0); _, _, Vt = np.linalg.svd(X); print(Vt)

[[ 0.70710678  0.70710678]
 [ 0.70710678 -0.70710678]]
```

Anexo: conceptos básicos y notación

Álgebra lineal: rama de las mates que estudia los espacios vectoriales y sus transformaciones lineales

Vector $\mathbf{x} \in \mathbb{R}^n$: suponemos que es vector columna, $\mathbf{x} = (x_1, x_2, \dots, x_n)^t$

- **Vector unitario** $\mathbf{e}_i$ o **onehot**: todo ceros salvo un uno en la posición i , $\mathbf{e}_i = (0, \dots, 0, 1, 0, \dots, 0)^t$

Matriz $\mathbf{A} \in \mathbb{R}^{m \times n}$: $\mathbf{A} = \begin{bmatrix} A_{11} & \cdots & A_{1n} \\ \vdots & \ddots & \vdots \\ A_{m1} & \cdots & A_{mn} \end{bmatrix}$ **cuadrada** si $m = n$

- **Matriz vista por filas y columnas:** $\mathbf{A} = [\mathbf{A}_{1,:}; \mathbf{A}_{2,:}; \dots; \mathbf{A}_{n,:}]$ $\mathbf{A} = [\mathbf{A}_{:,1}, \mathbf{A}_{:,2}, \dots, \mathbf{A}_{:,n}]$
- **Transpuesta:** $\mathbf{A}^t \in \mathbb{R}^{n \times m}$ con $(\mathbf{A}^t)_{ij} = A_{ji}$
- **Propiedades:** $(\mathbf{A}^t)^t = \mathbf{A}$ $(\mathbf{AB})^t = \mathbf{B}^t \mathbf{A}^t$ $(\mathbf{A} + \mathbf{B})^t = \mathbf{A}^t + \mathbf{B}^t$
- **Simétrica:** matriz cuadrada $\mathbf{A}$ tal que $\mathbf{A} = \mathbf{A}^t$
- **Reformato de matriz a vector:** $\text{vec}(\mathbf{A}) = [\mathbf{A}_{:,1}; \mathbf{A}_{:,2}; \dots; \mathbf{A}_{:,n}] \in \mathbb{R}^{mn \times 1}$
- **Reformato de vector a matriz:** puede ser por filas (python, C++) o por columnas (julia, octave, R, fortran)

Tensor: generalización de matriz a tres o más dimensiones

- **Orden o rango de un tensor:** dimensiones (concepto distinto en matrices)
- **Álgebra multilineal:** rama de las mates que generaliza los métodos del algebra lineal con tensores

Espacio vectorial: conjunto de **vectores** que pueden sumarse entre ellos y escalarse (multiplicarse) por **escalares**

Conjunto $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ linealmente independiente: ninguno puede representarse como combinación lineal del resto

Espacio generado o span de $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$: $\text{span}(\{\mathbf{x}_1, \dots, \mathbf{x}_n\}) = \{\mathbf{v} : \mathbf{v} = \sum_{i=1}^n \alpha_i \mathbf{x}_i, \alpha_i \in \mathbb{R}\}$

Base $\mathcal{B}$ de $\mathbb{R}^n$: genera $\mathbb{R}^n$; $\text{span}(\mathcal{B}) = \mathbb{R}^n$

- **Base estándar o canónica:** $\mathcal{B} = \{\mathbf{e}_1 = (1, 0, \dots, 0)^t, \dots, \mathbf{e}_n = (0, 0, \dots, 1)^t\}$

Transformación lineal: $f : \mathcal{V} \rightarrow \mathcal{W}$ tal que $f(\mathbf{v} + \mathbf{w}) = f(\mathbf{v}) + f(\mathbf{w})$ y $f(a\mathbf{v}) = af(\mathbf{v})$ para todo $\mathbf{v}, \mathbf{w} \in \mathcal{V}$

- **Representación matricial:** $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ con $\mathbf{A} \in \mathbb{R}^{m \times n}$ tal que $\mathbf{y} = \mathbf{A}\mathbf{x}$
- **Rango, espacio columna o imagen de $\mathbf{A}^{m \times n}$:** $\text{range}(\mathbf{A}) = \{\mathbf{v} \in \mathbb{R}^m : \mathbf{v} = \mathbf{A}\mathbf{x}, \mathbf{x} \in \mathbb{R}^n\}$
- **Espacio nulo o núcleo de $\mathbf{A}^{m \times n}$:** $\text{nullspace}(\mathbf{A}) = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} = \mathbf{0}\}$
- **Rango de $\mathbf{A} \in \mathbb{R}^{m \times n}$:** $\text{rank}(\mathbf{A})$ es la dimensión del espacio generado por las columnas de $\mathbf{A}$ (o filas; da igual)
- **Rango completo y deficiente:** **completo** si $\text{rank}(\mathbf{A}) = \min(m, n)$; **deficiente** si no

Norma de un vector $\mathbf{x}$: $\|\mathbf{x}\|$ es cualquier $f : \mathbb{R}^n \rightarrow \mathbb{R}$ tal que

$$f(\mathbf{x}) \geq 0 \quad f(\mathbf{x}) = 0 \Leftrightarrow \mathbf{x} = \mathbf{0} \quad f(t\mathbf{x}) = |t|f(\mathbf{x}) \quad f(\mathbf{x} + \mathbf{y}) \leq f(\mathbf{x}) + f(\mathbf{y})$$

- **Normas usuales:** $\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|$ $\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2} = \sqrt{\mathbf{x}^t \mathbf{x}}$ $\|\mathbf{x}\|_\infty = \max_i |x_i|$
- **Norma de Frobenius de $\mathbf{A} \in \mathbb{R}^{m \times n}$:** $\|\mathbf{A}\|_F = \|\text{vec}(\mathbf{A})\|_2 = \sqrt{\sum_{i=1}^m \sum_{j=1}^n A_{ij}^2}$

Traza de $\mathbf{A} \in \mathbb{R}^{n \times n}$: $\text{tr}(\mathbf{A}) = \sum_{i=1}^n A_{ii}$

- **Propiedades:** $\text{tr}(\mathbf{A}) = \text{tr}(\mathbf{A}^t)$, $\text{tr}(\mathbf{A} + \mathbf{B}) = \text{tr}(\mathbf{A}) + \text{tr}(\mathbf{B})$, $\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$, $\text{tr}(\mathbf{A}) = \sum_i \lambda_i$
- **Truco de la traza:** $\mathbf{x}^t \mathbf{A} \mathbf{x} = \text{tr}(\mathbf{x}^t \mathbf{A} \mathbf{x}) = \text{tr}(\mathbf{x} \mathbf{x}^t \mathbf{A})$

Determinante de $\mathbf{A} \in \mathbb{R}^{n \times n}$: $|\mathbf{A}|$ mide cuánto cambia un volumen unitario viendo $\mathbf{A}$ como una transformación lineal

- **Propiedades:** $|\mathbf{A}| = |\mathbf{A}^t|$ $|c\mathbf{A}| = c^n |\mathbf{A}|$ $|\mathbf{AB}| = |\mathbf{A}| |\mathbf{B}|$ $|\mathbf{A}| = 0$ sii $\mathbf{A}$ singular $|\mathbf{A}| = \prod_i \lambda_i$

Número de condición de $\mathbf{A}$: $\kappa(\mathbf{A})$ mide la inestabilidad numérica de los cálculos con $\mathbf{A}$ (mejor que el determinante)

$$\kappa(\mathbf{A}) = \|\mathbf{A}\| \|\mathbf{A}^{-1}\| \quad \kappa(\mathbf{A}) \geq 1$$

- **$\mathbf{A}$ bien condicionada:** si $\kappa(\mathbf{A}) \approx 1$; si no, $\mathbf{A}$ está **mal condicionada** y, si $\kappa(\mathbf{A})$ es grande, $\mathbf{A}$ es **casi-singular**

- **Número de condición con L2:** $\kappa(\mathbf{A}) = \frac{\sigma_{\max}}{\sigma_{\min}} = \sqrt{\frac{\lambda_{\max}}{\lambda_{\min}}}$ σ valor singular, λ valor propio

Matriz diagonal: $\mathbf{D} = \text{diag}(d_1, \dots, d_n) \in \mathbb{R}^{n \times n}$ con ceros fuera de la diagonal

- **Construcción y extracción de una matriz diagonal:** $\mathbf{D} = \text{diag}(\mathbf{d})$ y $\mathbf{d} = \text{diag}(\mathbf{D})$

Matriz identidad: $\mathbf{I} = \text{diag}(1, \dots, 1) \in \mathbb{R}^{n \times n}$ con unos en diagonal y ceros fuera

Matriz diagonal por bloques: $[\mathbf{A}, \mathbf{0}; \mathbf{0}, \mathbf{B}]$

Matriz banda-diagonal: con elementos no nulos en una banda de ancho k centrada en la diagonal; **tridiagonal** si $k = 3$

Matriz triangular superior/inferior: solo tiene elementos no nulos en la diagonal y por encima/debajo

- **Propiedades:** valores propios en la diagonal, por lo que $|\mathbf{A}| = \prod_i A_{ii}$

Forma cuadrática asociada a una matriz real y simétrica, $\mathbf{A} \in \mathbb{R}^{n \times n}$: $f(\mathbf{x}) = \mathbf{x}^t \mathbf{A} \mathbf{x} = \sum_i \sum_j A_{ij} x_i x_j$

- **A definida positiva** ($\mathbf{A} \succ 0$ o $\mathbf{A} > 0$) sii $f(\mathbf{x}) > 0$ para todo $\mathbf{x}$ no nulo
- **A semi-definida positiva** ($\mathbf{A} \succeq 0$ o $\mathbf{A} \geq 0$) sii $f(\mathbf{x}) \geq 0$ para todo $\mathbf{x}$ no nulo
- **A definida negativa** ($\mathbf{A} \prec 0$ o $\mathbf{A} < 0$) sii $f(\mathbf{x}) < 0$ para todo $\mathbf{x}$ no nulo
- **A semi-definida negativa** ($\mathbf{A} \preceq 0$ o $\mathbf{A} \leq 0$) sii $f(\mathbf{x}) \leq 0$ para todo $\mathbf{x}$ no nulo
- **A indefinida** si no es semi-definida positiva ni semi-definida negativa
- **Relación entre A y -A:** $\mathbf{A} \succ 0$ sii $-\mathbf{A} \prec 0$ $\mathbf{A} \succeq 0$ sii $-\mathbf{A} \preceq 0$ $\mathbf{A}$ indefinida sii $-\mathbf{A}$ indefinida
- **Condición necesaria y suficiente para $\mathbf{A} \succ 0$:** valores propios positivos

Vectores ortogonales: $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ son ortogonales si $\mathbf{x}^t \mathbf{y} = 0$

Vector normalizado: $\mathbf{x} \in \mathbb{R}^n$ está normalizado si $\|\mathbf{x}\|_2 = 1$

Conjunto de vectores ortonormal: si son ortogonales dos a dos y normalizados

Matriz ortogonal $\mathbf{U} \in \mathbb{R}^{n \times n}$: si el conjunto de sus columnas (o filas) es ortonormal

- **Caracterización de matriz ortogonal:** $\mathbf{U}$ ortogonal sii $\mathbf{U}^t \mathbf{U} = \mathbf{I} = \mathbf{U} \mathbf{U}^t$ sii $\mathbf{U}^t = \mathbf{U}^{-1}$
- **Propiedades de las transformaciones ortogonales:** preservan longitudes y ángulos

$$\|\mathbf{U} \mathbf{x}\|_2 = \|\mathbf{x}\|_2 \quad \cos(\alpha(\mathbf{U} \mathbf{x}, \mathbf{U} \mathbf{y})) = \frac{(\mathbf{U} \mathbf{x})^t (\mathbf{U} \mathbf{y})}{\|\mathbf{U} \mathbf{x}\| \|\mathbf{U} \mathbf{y}\|} = \frac{\mathbf{x}^t \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|} = \cos(\alpha(\mathbf{x}, \mathbf{y}))$$

Producto escalar: $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n, \quad \langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^t \mathbf{y} = \sum_{i=1}^n x_i y_i = \mathbf{y}^t \mathbf{x}$

Producto vectorial: $\mathbf{x} \in \mathbb{R}^m, \mathbf{y} \in \mathbb{R}^n, \quad \mathbf{x} \mathbf{y}^t = \begin{bmatrix} x_1 y_1 & \cdots & x_1 y_n \\ \vdots & \ddots & \vdots \\ x_m y_1 & \cdots & x_m y_n \end{bmatrix} \in \mathbb{R}^{m \times n}$

Producto matriz-vector: $\mathbf{A} \in \mathbb{R}^{m \times n}, \mathbf{x} \in \mathbb{R}^n, \quad \mathbf{y} = \mathbf{A} \mathbf{x} \in \mathbb{R}^m$

• **Interpretaciones:** $\mathbf{y} = \begin{bmatrix} \mathbf{a}_1^t \\ \vdots \\ \mathbf{a}_m^t \end{bmatrix} \mathbf{x} = \begin{bmatrix} \mathbf{a}_1^t \mathbf{x} \\ \vdots \\ \mathbf{a}_m^t \mathbf{x} \end{bmatrix} \quad \mathbf{y} = [\mathbf{a}_1, \dots, \mathbf{a}_n] \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \mathbf{a}_1 x_1 + \cdots + \mathbf{a}_n x_n$

Producto de matrices: $\mathbf{A} \in \mathbb{R}^{m \times n}, \mathbf{B} \in \mathbb{R}^{n \times p}, \quad \mathbf{C} = \mathbf{A} \mathbf{B} \in \mathbb{R}^{m \times p} \quad \text{con} \quad C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}$

• **Propiedades:** $(\mathbf{A} \mathbf{B}) \mathbf{C} = \mathbf{A} (\mathbf{B} \mathbf{C}) \quad \mathbf{A} (\mathbf{B} + \mathbf{C}) = \mathbf{A} \mathbf{B} + \mathbf{A} \mathbf{C} \quad \mathbf{A} \mathbf{B} \neq \mathbf{B} \mathbf{A}$

• **Interpretaciones:** 4; fila-columna, columna-fila, matriz-columna y fila-matriz

$$\mathbf{C} = \begin{bmatrix} \mathbf{a}_1^t \\ \vdots \\ \mathbf{a}_m^t \end{bmatrix} [\mathbf{b}_1, \dots, \mathbf{b}_p] = \begin{bmatrix} \mathbf{a}_1^t \mathbf{b}_1 & \cdots & \mathbf{a}_1^t \mathbf{b}_p \\ \vdots & \ddots & \vdots \\ \mathbf{a}_m^t \mathbf{b}_1 & \cdots & \mathbf{a}_m^t \mathbf{b}_p \end{bmatrix} \quad \mathbf{C} = [\mathbf{a}_1, \dots, \mathbf{a}_n] \begin{bmatrix} \mathbf{b}_1^t \\ \vdots \\ \mathbf{b}_n^t \end{bmatrix} = \sum_{i=1}^n \mathbf{a}_i \mathbf{b}_i^t$$

$$\mathbf{C} = \mathbf{A} [\mathbf{b}_1, \dots, \mathbf{b}_p] = [\mathbf{A} \mathbf{b}_1, \dots, \mathbf{A} \mathbf{b}_p] \quad \mathbf{C} = \begin{bmatrix} \mathbf{a}_1^t \\ \vdots \\ \mathbf{a}_m^t \end{bmatrix} \mathbf{B} = \begin{bmatrix} \mathbf{a}_1^t \mathbf{B} \\ \vdots \\ \mathbf{a}_m^t \mathbf{B} \end{bmatrix}$$

Inversa de $\mathbf{A} \in \mathbb{R}^{n \times n}$: matriz $\mathbf{A}^{-1}$ tal que $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I} = \mathbf{A}\mathbf{A}^{-1}$

- **Existencia:** $\mathbf{A}^{-1}$ existe sii $|\mathbf{A}| \neq 0$; esto es, si $\mathbf{A}$ es no singular
- **Propiedades:** $(\mathbf{A}^{-1})^{-1} = \mathbf{A}$ $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$ $(\mathbf{A}^{-1})^t = (\mathbf{A}^t)^{-1} = \mathbf{A}^{-t}$
- **Inversa de $\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$:** $\mathbf{A}^{-1} = \frac{1}{|\mathbf{A}|} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$
- **Inversa de una diagonal por bloques:** $\begin{pmatrix} \mathbf{A} & \mathbf{0} \\ \mathbf{0} & \mathbf{B} \end{pmatrix}^{-1} = \begin{pmatrix} \mathbf{A}^{-1} & \mathbf{0} \\ \mathbf{0} & \mathbf{B}^{-1} \end{pmatrix}$

Descomposición propia (EVD): de una matriz cuadrada $\mathbf{A} \in \mathbb{R}^{n \times n}$

- **Valor y vector propio:** $\lambda \in \mathbb{R}$ es un **valor propio** y $\mathbf{u} \in \mathbb{R}^n$ "su" **vector propio** asociado si $\mathbf{A}\mathbf{u} = \lambda\mathbf{u}$, $\mathbf{u} \neq \mathbf{0}$
- **No unicidad del vector propio:** si $\mathbf{A}\mathbf{u} = \lambda\mathbf{u}$, $\mathbf{A}(c\mathbf{u}) = c\mathbf{A}\mathbf{u} = c\lambda\mathbf{u} = \lambda(c\mathbf{u})$ para todo $c \in \mathbb{R}$
- **Asunción de normalidad:** por simplicidad, se asume que "el" vector propio asociado a λ está normalizado, $\|\mathbf{u}\|_2 = 1$; el entrecomillado "el" se debe a que $-\mathbf{u}$ también está normalizado, por lo que tenemos dos opciones
- **Ecuación característica:** $|\lambda\mathbf{I}_n - \mathbf{A}| = 0$ para hallar los n valores y vectores propios de $\mathbf{A}$
- **Propiedades:** $\text{tr}(\mathbf{A}) = \sum_{i=1}^n \lambda_i$ $|\mathbf{A}| = \prod_{i=1}^n \lambda_i$ $\text{rank}(\mathbf{A}) = \sum_{i=1}^n \mathbb{I}(\lambda_i > 0)$
- **Ecuación global:** $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n)$ y $\mathbf{U} = (\mathbf{u}_1, \dots, \mathbf{u}_n)$ son matrices de valores y vectores propios de $\mathbf{A}$ si:

$$\mathbf{A}\mathbf{U} = [\mathbf{A}\mathbf{u}_1, \mathbf{A}\mathbf{u}_2, \dots, \mathbf{A}\mathbf{u}_n] = [\lambda_1\mathbf{u}_1, \lambda_2\mathbf{u}_2, \dots, \lambda_n\mathbf{u}_n] = \mathbf{U}\mathbf{\Lambda}$$

- **Descomposición propia (EVD):** si $\mathbf{U}$ ortogonal, $\mathbf{A} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^{-1}$

EVD de una matriz simétrica: sea $\mathbf{A} \in \mathbb{R}^{n \times n}$ simétrica

- **Valores y vectores propios:** $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n)$, $\lambda_i \in \mathbb{R}$; $\mathbf{U}$ ortogonal, $\mathbf{U}^t \mathbf{U} = \mathbf{U} \mathbf{U}^t = \mathbf{I}$

- **EVD:** $\mathbf{A} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^t = (\mathbf{u}_1, \dots, \mathbf{u}_n) \text{diag}(\lambda_1, \dots, \lambda_n) (\mathbf{u}_1^t; \dots; \mathbf{u}_n^t) = \sum_{i=1}^n \lambda_i \mathbf{u}_i \mathbf{u}_i^t$

- **Inversa (si existe):** $\mathbf{A}^{-1} = \mathbf{U} \mathbf{\Lambda}^{-1} \mathbf{U}^t = \sum_{i=1}^n \frac{1}{\lambda_i} \mathbf{u}_i \mathbf{u}_i^t$

- **Forma cuadrática asociada a $\mathbf{A}$:** $f(\mathbf{x}) = \mathbf{x}^t \mathbf{A} \mathbf{x} = \mathbf{x}^t \mathbf{U} \mathbf{\Lambda} \mathbf{U}^t \mathbf{x} \stackrel{\mathbf{y}=\mathbf{U}^t \mathbf{x}}{=} \mathbf{y}^t \mathbf{\Lambda} \mathbf{y} = \sum_{i=1}^n \lambda_i y_i^2$

- **$\mathbf{A}$ definida positiva** sii $\lambda_i > 0$ para todo i
- **$\mathbf{A}$ semidefinida positiva** sii $\lambda_i \geq 0$ para todo i
- **$\mathbf{A}$ definida negativa** sii $\lambda_i < 0$ para todo i
- **$\mathbf{A}$ semidefinida negativa** sii $\lambda_i \leq 0$ para todo i
- **$\mathbf{A}$ indefinida** sii tiene λ_i positivos y negativos

Matriz de covarianzas: $\mathbf{\Sigma} \in \mathbb{R}^{D \times D}$ simétrica y semi-definida positiva sii $\mathbf{\Sigma}$ es matriz de covarianzas

- **Valores propios:** no negativos; positivos si $\mathbf{\Sigma}$ es definida positiva, por lo que $\mathbf{\Sigma}^{-1}$ existe y es definida positiva
- **Distancia de Mahalanobis (al cuadrado):** forma cuadrática asociada a $\mathbf{\Sigma}^{-1}$ definida positiva

$$f(\mathbf{x}) = \mathbf{x}^t \mathbf{\Sigma}^{-1} \mathbf{x} = \mathbf{x}^t \mathbf{U} \mathbf{\Lambda} \mathbf{U}^t \mathbf{x} \stackrel{\mathbf{y}=\mathbf{U}^t \mathbf{x}}{=} \mathbf{y}^t \mathbf{\Lambda} \mathbf{y} = \sum_{d=1}^D \lambda_d y_d^2$$

Ejemplo: distancia de Mahalanobis al origen (al cuadrado) con $\Sigma = \begin{bmatrix} \frac{5}{2} & \frac{3}{2} \\ \frac{3}{2} & \frac{5}{2} \end{bmatrix}$

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt
S = np.array([[5/2, 3/2], [3/2, 5/2]])
La, U = np.linalg.eigh(S); i = La.argsort()[::-1]; La = La[i]; U = U[:,i]
I = U @ np.diag(1/La) @ U.T; print(La, "\n", U, "\n", I); x_min = y_min = -2.2; x_max = y_max = 2.2
X, Y = np.meshgrid(np.linspace(x_min, x_max, num=64), np.linspace(y_min, y_max, num=64))
XY = np.c_[np.ravel(X), np.ravel(Y)]; d = lambda xy: xy.T @ I @ xy; D = np.apply_along_axis(d, 1, XY)
fig, ax = plt.subplots(figsize=(4, 4))
ax.set(aspect='equal', xlim=(x_min, x_max), ylim=(y_min, y_max))
ax.grid(); ax.axhline(0, color='black'); ax.axvline(0, color='black')
ax.contour(X, Y, D.reshape(X.shape), 10, colors='orange', linestyle='solid')
cp = ax.contourf(X, Y, D.reshape(X.shape), 10, cmap="Blues")
plt.colorbar(cp, ax=ax, shrink=0.8);
```

```
[4. 1.]
[[ 0.70710678 -0.70710678]
 [ 0.70710678  0.70710678]]
[[ 0.625 -0.375]
 [-0.375  0.625]]
```

